

WorkHub — Design Note (Phase 1)

1. Architecture and Layering

WorkHub follows a layered Spring Boot architecture:

- **Controller Layer**
Handles HTTP requests and responses using DTOs.
- **Service Layer**
Contains business logic and defines transaction boundaries using `@Transactional`.
- **Repository Layer**
Uses Spring Data JPA for database operations.
- **Entity Layer**
Represents database tables and includes `tenantId` for multi-tenancy.
- **DTO Layer**
Used for request validation and API communication.
- **Configuration Layer**
Handles security and JWT logic.

Error Handling

Errors are handled using `GlobalExceptionHandler` and returned in a consistent JSON format (`ApiError`) including status, message, and validation details.

2. Multi-Tenancy Approach

The system uses a **shared database with a `tenantId` column** to separate data between tenants.

Tenant Resolution

- **TenantFilter** reads `X-Tenant-ID` from the request and stores it in `TenantContext`.
- After login, the **JWT token contains `tenantId`**, and `JwtAuthenticationFilter` extracts it and sets it in `TenantContext`.
- If both header and token exist, the **JWT value is used**.

Phase 1 Scope

Tenant ID is:

- assigned during registration
 - stored inside JWT
 - available in authenticated requests
-

3. Security Model

- Authentication is implemented using **JWT** (stateless).
- No server-side session is used (`SessionCreationPolicy.STATELESS`).

Access Control

- Public endpoints:
 - `/auth/login`
 - `/auth/register`
 - `/h2-console`
- All other endpoints require a valid JWT.

Roles

- `TENANT_ADMIN`
- `TENANT_USER`

Example of authorization:

```
@PreAuthorize("hasRole('TENANT_ADMIN')")
```

4. Transaction Boundary and Rollback

Transactions are defined in the service layer using:

```
@Transactional
```

Use Case: User Registration

- A user is created and saved with a `tenantId`.
- If an exception occurs after saving, the transaction is rolled back.

Rollback Demonstration

- If email contains "fail" → exception is thrown → rollback happens.
- `registerWithFailure()` always throws an exception after save → ensures rollback.

Result

- No partial data is saved
- Database consistency is maintained